

RAG Engine

Moteur RAG (Retrieval-Augmented Generation) production-ready, conçu pour indexer et interroger une base de connaissances documentaire via recherche vectorielle sémantique.

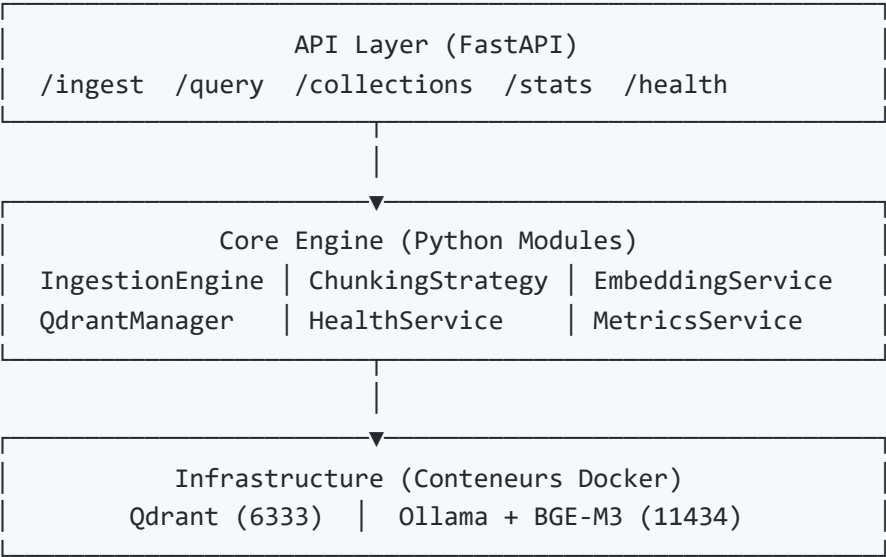
Infrastructure : Ce projet tourne sur une architecture locale auto-hébergée.

Le code source est disponible ici (GitHub, accès sur invitation).

Le pipeline CI/CD est géré par un runner GitLab auto-hébergé (on-premise).

Composant	Technologie	Rôle
Conversion documentaire	Docling (IBM)	PDF / HTML / DOCX → Markdown structuré
Embedding	BGE-M3 via Ollama	Vecteurs multilingues 1024 dimensions
Base vectorielle	Qdrant	Stockage, filtrage par domaine, index HNSW
Chunking	LlamaIndex	Découpage intelligent Markdown-aware
Évaluation qualité	Ragas	Métriques context_precision, faithfulness...
API	FastAPI	REST endpoints : ingestion, query, monitoring

Architecture



```
rag-engine/
├── api/                # FastAPI : routes, modèles Pydantic
├── cli/                # CLI management (ingest, query, health...)
├── core/
│   ├── config.py       # Chargement YAML + overrides env
│   ├── ingestion/      # Engine, chunking, processors (Docling)
│   ├── vector_store/   # QdrantManager, EmbeddingService
│   └── monitoring/     # Health, metrics, quality (Ragas)
├── config/             # settings.yaml, domains.yaml
├── scripts/
│   ├── ollama-entrypoint.sh    # Entrypoint Ollama (auto-pull BGE-M3)
│   └── openwebui_rag_tool.py   # Tool OpenWebUI pour connecter un LLM
├── tests/
│   ├── unit/            # Tests isolés (mocks, pas de réseau)
│   ├── integration/     # Tests E2E (Qdrant + Ollama requis)
│   └── quality/         # Évaluation Ragas
├── docker-compose.yml
├── Dockerfile
└── .gitlab-ci.yml
```

État actuel

Le moteur est **fonctionnel en local** — l'ingestion et la recherche vectorielle tournent en production sur l'infra Docker.

Ingestions réalisées

Collection	Source	Vecteurs	Format	Détail
kb_n8n	Documentation n8n complète	7 157	Markdown (.md)	~200 fichiers, docs officielles
kb_cv	CV PDF (Yassir Boulouiha)	15	PDF → Markdown via Docling	Conversion automatique OCR+tables

Connexion LLM (en cours)

Le moteur est actuellement une **base vectorielle pure** : il récupère les chunks les plus pertinents via `/query` mais ne génère pas de réponse. La connexion à un LLM est testée via **OpenWebUI** :

- Un conteneur [OpenWebUI](#) est déployé en parallèle
- L'API Gemini (Google) est connectée comme provider LLM
- Un **Tool OpenWebUI** (`scripts/openwebui_rag_tool.py`) appelle l'endpoint `/query` du RAG Engine pour injecter les contextes dans le prompt du LLM

Ce Tool permet au LLM de rechercher automatiquement dans `kb_n8n` ou `kb_cv` selon la question posée, transformant le moteur de recherche vectorielle en assistant RAG complet.

Installation

Prérequis

- Docker & Docker Compose
- (Optionnel) Python 3.11+ pour le développement local

Déploiement complet — une seule commande

Le `docker-compose.yml` orchestre les 3 services : **Qdrant**, **Ollama + BGE-M3** et l'**API FastAPI**.

Le `Dockerfile` build automatiquement l'image de l'API au premier lancement.

```
git clone <repo-url> && cd rag-engine
docker-compose up -d --build
```

Au premier lancement, Ollama télécharge automatiquement le modèle BGE-M3 (~2 min).
L'API attend que Qdrant et Ollama soient `healthy` avant de démarrer (`depends_on:` `condition: service_healthy`).

```
# Vérifier que tout tourne
docker-compose ps

# Vérifier que le modèle est chargé
docker exec rag-ollama ollama list

# Tester l'API
curl http://localhost:8000/health
```

Service	Conteneur	Port	Rôle
qdrant	rag-qdrant	6333, 6334	Base vectorielle
ollama	rag-ollama	11434	Modèle d'embedding BGE-M3
rag_engine	rag-engine-api	8000	API FastAPI

Développement local (hot-reload)

Pour itérer rapidement sur le code sans rebuild l'image Docker à chaque changement :

```
# 1. Lancer uniquement l'infra
docker-compose up -d qdrant ollama

# 2. Installer les dépendances Python
python -m venv venv && venv\Scripts\activate # Windows
pip install -r requirements.txt

# 3. Lancer l'API avec hot-reload
python main.py api
# ou : uvicorn api.main:app --reload --host 0.0.0.0 --port 8000

# 4. Ou utiliser le CLI directement
python main.py cli health
python main.py cli ingest ./documents --domain n8n
```

Note : En mode dev local, `settings.yaml` pointe sur `localhost` pour Qdrant et Ollama.

En Docker complet, les variables d'environnement `QDRANT_HOST=qdrant` et `OLLAMA_URL=http://ollama:11434` remplacent automatiquement les valeurs du fichier de config.

Utilisation

CLI

```
# Ingérer un dossier de documents
python main.py cli ingest ./documents --domain n8n

# Rechercher dans la base
python main.py cli query "comment gérer les erreurs" -c kb_n8n

# Lister les collections
python main.py cli collections

# Vérifier la santé des services
python main.py cli health

# Statistiques globales
python main.py cli stats
```

API REST

```
# Ingestion synchrone
curl -X POST http://localhost:8000/ingest/folder/sync \
  -H "Content-Type: application/json" \
  -d '{"folder_path": "./documents", "domain": "n8n"}'

# Recherche
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" \
  -d '{"query": "comment faire X", "collections": ["kb_n8n"], "top_k": 5}'

# Health check
curl http://localhost:8000/health

# Statistiques
curl http://localhost:8000/stats
```

Endpoints API

Endpoint	Méthode	Description
/health	GET	Health check (Qdrant + Ollama)
/stats	GET	Statistiques globales
/collections	GET	Liste des collections
/collections/{name}	GET / DELETE	Détail / suppression collection
/ingest/folder	POST	Ingestion async dossier
/ingest/folder/sync	POST	Ingestion sync dossier
/ingest/file	POST	Ingestion fichier unique
/query	POST	Recherche RAG multi-collections
/query/search/{collection}	GET	Recherche simple
/evaluate	POST	Évaluation qualité Ragas
/docs	GET	Documentation Swagger

Connexion LLM via OpenWebUI

Le RAG Engine ne génère pas de réponse lui-même — il fournit les **contextes pertinents** à un LLM externe. Le script `scripts/openwebui_rag_tool.py` implémente un **Tool** OpenWebUI qui fait le pont :

```
Utilisateur → OpenWebUI → LLM (Gemini / GPT / Ollama)
                    ↓ tool_call
                RAG Engine /query
                    ↓
        Contextes injectés dans le prompt
                    ↓
                Réponse augmentée
```

Déployer OpenWebUI

```
docker run -d -p 3000:8080 \
  --name open-webui \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  ghcr.io/open-webui/open-webui:v0.6.5
```

Installer le Tool

1. Ouvrir <http://localhost:3000>
2. **Workspace** → **Outils** → + → coller le contenu de `scripts/openwebui_rag_tool.py`
3. **Workspace** → **Modèles** → assigner le tool au modèle LLM souhaité

Le tool expose deux fonctions :

- `search_n8n_docs(query)` → recherche dans `kb_n8n`
 - `search_cv(query)` → recherche dans `kb_cv`
-

Tests

Stratégie

Catégorie	Dossier	Dépendances externes	CI/CD
Unitaires	tests/unit/	Aucune (tout mocké)	[OUI] Obligatoire
Intégration	tests/integration/	Qdrant + Ollama	[NON] Optionnel
Qualité	tests/quality/	Qdrant + Ollama + Ragas	[NON] Manuel

Les tests unitaires mockent systématiquement Qdrant et Ollama pour garantir une exécution rapide et déterministe, sans aucune infrastructure.

Exécution

```
# Tests unitaires (dev quotidien, ~10s, aucune infra requise)
pytest tests/unit/ -v

# Tests d'intégration (nécessite docker-compose up)
pytest tests/integration/ -v

# Tout sauf les tests lents
pytest -m "not slow"

# Avec couverture
pytest tests/unit/ --cov=core --cov=api --cov-report=term
```

Couverture des tests unitaires

Fichier	Composant testé
test_models.py	Modèles Pydantic (validation, sérialisation)
test_config.py	Chargement configuration YAML
test_chunking.py	Stratégies de chunking LlamaIndex
test_processors.py	Factory et processors de fichiers
test_embeddings.py	EmbeddingService (mock Ollama)
test_monitoring.py	HealthService, MetricsService, RAGQualityMonitor
test_error_handling.py	Gestion d'erreurs (ingestion, config, Qdrant)
test_cli.py	Commandes CLI

CI/CD (GitLab auto-hébergé)

Le pipeline CI/CD tourne sur un runner **GitLab auto-hébergé** (on-premise), structuré en 4 stages :

setup → test → quality → deploy

Job	Stage	Déclencheur	Bloquant
unit_tests	test	Chaque push	[OUI]
integration_tests	test	master / develop	[NON]
code_quality	quality	Chaque push	[NON]
deploy_staging	deploy	Manuel	—

- **unit_tests** : 121 tests, ~18s, aucune infra requise (tout mocké via `unittest.mock`).
- **integration_tests** : nécessite Qdrant + Ollama actifs sur le runner.
- **code_quality** : lint flake8.
- **deploy_staging** : déploiement manuel via `docker-compose up -d --build` .

Architecture de déploiement : le code source est mirrored sur GitHub (privé) et sur un GitLab local.

Le CI/CD s'exécute uniquement sur le GitLab on-premise — GitHub Actions n'est pas utilisé (fonctionnalité payante pour les repos privés).

Évaluation qualité (Ragas)

L'évaluation qualité est **découplée du pipeline CI/CD** et se déclenche manuellement sur les documents déjà indexés dans Qdrant. Elle nécessite Qdrant et Ollama actifs.

```
pytest tests/quality/ragas_eval.py -v
```

Métriques évaluées

Métrique	Seuil	Ce qu'elle détecte
context_precision	≥ 0.70	Bruit dans les chunks récupérés
context_recall	≥ 0.60	Perte d'information au chunking
faithfulness	≥ 0.80	Hallucinations du LLM
answer_relevancy	≥ 0.75	Réponses hors-sujet

Note : Ragas nécessite un LLM pour évaluer `faithfulness` et `answer_relevancy` .
L'évaluation complète sera possible une fois la connexion LLM finalisée via OpenWebUI.

Configuration

Fichier principal : `config/settings.yaml`

```
qdrant:
  host: "localhost"
  port: 6333
  vector_size: 1024      # BGE-M3 dimension

ollama:
  base_url: "http://localhost:11434"
  embedding_model: "bge-m3"

ingestion:
  chunk_size: 512
  chunk_overlap: 50
  supported_extensions: [".pdf", ".html", ".docx", ".md"]
```

Perspectives d'évolution

Gate qualité à l'ingestion (*priorité haute*)

Actuellement, tout document soumis à `/ingest` est accepté sans évaluation. La prochaine étape est d'intégrer Ragas comme **gate de validation** avant indexation dans Qdrant :

[Document entrant] → [Chunking] → [Ragas spot-check] → [OK] Indexé | [FAIL] Refusé

Un document qui génère des chunks trop bruités (`context_precision < 0.70`) ou cause des hallucinations (`faithfulness < 0.80`) serait mis en quarantaine avec un rapport détaillé plutôt qu'indexé silencieusement.

Connexion LLM native

Le moteur est actuellement une base vectorielle pure, la connexion LLM passe par OpenWebUI en externe. Deux options sont envisagées pour une intégration native :

Option	Cas d'usage	Avantage
API externe (Gemini, OpenAI, Mistral...)	Démo, VRAM limitée	Aucune contrainte matérielle
LLM local via Ollama (Qwen, Llama...)	Production on-premise	Données privées, 0 coût

Une fois connecté, l'évaluation Ragas complète devient possible et les métriques `faithfulness` / `answer_relevancy` sont calculables sur la base existante.

Autres perspectives

- Tests d'intégration bloquants en CI (Docker-in-Docker)
 - Génération automatique de golden samples Ragas depuis les nouveaux docs
 - Exposition des scores qualité via `/stats` pour monitoring continu
 - Multi-collection routing automatique selon le domaine détecté
-

Documentation complémentaire

Les documents détaillés suivants existent dans le dépôt privé (non exposés ici) :

- ARCHITECTURE.md : Architecture technique, diagrammes Mermaid
- TECHNICAL_DOCUMENTATION.md : Guide complet d'utilisation
- API_REFERENCE.md : Référence complète de l'API REST
- DEVELOPMENT_GUIDE.md : Guide de contribution

Document PDF généré via le paquet npm md-to-pdf (conversion Markdown → HTML → PDF) pour garantir une fidélité visuelle optimale par rapport à la documentation d'origine.

License

Projet personnel — Tous droits réservés.

© 2025-2026 Yassir Boulouiha Gnaoui